

HIPERSALES

# RELATÓRIO COMPLETO DE ANÁLISE DO SISTEMA

Funcionalidade, segurança, arquitetura e plano de melhoria

AUTORIA

**Miguel Alameida**

Desenvolvedor Nível 3

DATA DA ANÁLISE

6 de outubro de 2026

CLASSIFICAÇÃO

Uso interno e confidencial

# 1. Controle do documento

|               |                                                             |
|---------------|-------------------------------------------------------------|
| DOCUMENTO     | Relatório completo de análise do sistema HiperSales         |
| AUTOR         | Miguel Alameida                                             |
| CARGO         | Desenvolvedor Nível 3                                       |
| DATA          | 6 de outubro de 2026                                        |
| CLASSIFICAÇÃO | Uso interno e confidencial                                  |
| ESCOPO        | Código-fonte, banco SQLite, frontend PWA e serviço WhatsApp |
| MÉTODO        | Revisão estática e verificações locais não destrutivas      |

## Nota importante

Nenhum arquivo funcional foi alterado e o servidor não foi iniciado. A inicialização do backend executa migrações, inicia rotinas automáticas e redefine a credencial fixa do superadministrador.

## Objetivo

Este relatório documenta o funcionamento do HiperSales, avalia sua arquitetura e segurança, identifica riscos técnicos e operacionais e propõe uma sequência prática de correções para elevar o sistema a um padrão seguro de produção e operação multiempresa.

## Limitações

- Não foi realizado teste de invasão ativo contra ambiente publicado.
- Não foram usados dados pessoais individuais nem exibidas credenciais ou tokens encontrados.
- Riscos de dependências refletem o resultado do npm audit em 6 de outubro de 2026.

## 2. Resumo executivo

### Avaliação geral: risco alto

O sistema entrega o fluxo comercial esperado e possui controles básicos corretos, mas não deve ser exposto publicamente como solução multiempresa antes da correção das credenciais fixas, anexos executáveis e falhas de isolamento entre tenants.

A aplicação começou como um sistema local de uma empresa e recebeu posteriormente recursos de multi-tenancy. Essa evolução é visível na boa cobertura de tenant\_id nas consultas principais, porém os caminhos legados de configurações, relatórios, notificações e WhatsApp ainda assumem o tenant 1 ou usam destinatários globais.

### Síntese dos riscos

|           |   |                                                                         |
|-----------|---|-------------------------------------------------------------------------|
| CRÍTICA   | 3 | Comprometimento administrativo ou vazamento entre empresas              |
| ALTA      | 3 | Tomada de conta, exposição de segredos ou componente vulnerável         |
| MÉDIA     | 5 | Defesas incompletas, indisponibilidade ou injeção de conteúdo           |
| FUNCIONAL | 5 | Resultados incorretos, baixa manutenibilidade ou operação inconsistente |

### Decisão recomendada

- Manter o sistema restrito a rede ou acesso controlado até concluir os itens críticos.
- Executar um hotfix de segurança antes de adicionar novas funcionalidades.
- Realizar backup consistente do SQLite e iniciar controle de versão antes das alterações.
- Tratar a correção de multi-tenancy como projeto de dados e autorização, não apenas como filtros adicionais.

## 3. Arquitetura e funcionamento

### Fluxo principal

#### Navegador

SPA/PWA em HTML, CSS e JavaScript puro. A interface utiliza fetch com cookie de sessão e mantém o estado da aplicação em memória.

#### Backend

Servidor Python baseado em ThreadingHTTPServer. O mesmo processo entrega arquivos estáticos, implementa a API REST, gera PDFs e executa schedulers em threads daemon.

#### Persistência

SQLite em modo WAL, com migrações e seed executados na inicialização. Sessões, configurações, anexos e fila de mensagens ficam no mesmo banco.

#### Integrações

SMTP, APIs públicas de CNPJ, Evolution API e um serviço Node/Baileys separado que consulta endpoints internos do backend.

### Pontos de entrada

- backend/app.py: servidor, banco, autenticação, regras de negócio, PDFs e schedulers.
- frontend/app.js: interface completa, navegação, formulários e renderização.
- whatsapp-service/index.mjs: conexão Baileys e consumo da fila interna.
- scripts/import\_carga\_leon.py: importação de planilha, usuários e vínculos comerciais.

## 4. Funcionalidades identificadas

---

- Autenticação por cookie e perfis superadministrador, administrador e representante comercial.
- Criação de tenants e administração de usuários por empresa.
- Cadastro, consulta, edição, inativação e associação de clientes.
- Cadastro de empresas representadas, produtos e importação/exportação de catálogo.
- Propostas e pedidos com itens, valores negociados, descontos, comissão, frete e pagamento.
- Fluxo de status, linha do tempo, PDFs e notificações de pedidos.
- Registro de ocorrências com anexos, parecer administrativo e retenção temporária.
- Metas comerciais, acompanhamento de carteira, dashboards e relatórios gerenciais.
- Consulta automática de CNPJ em dois provedores públicos.
- Configuração de SMTP, templates de mensagens e WhatsApp.
- PWA com manifest, service worker e cache de recursos estáticos.
- Rotinas automáticas para relatórios, lembretes de metas e respostas de WhatsApp.

### Perfis e autorização

O representante visualiza sua carteira, empresas associadas, pedidos e ocorrências. O administrador opera os dados do tenant, usuários, produtos, clientes, pedidos, metas e configurações. O superadministrador cria e acompanha tenants. A autorização é aplicada no backend, mas possui exceções legadas descritas nos achados.

## 5. Controles positivos observados

---

- Consultas SQL majoritariamente parametrizadas, reduzindo significativamente o risco de SQL injection.
- Tokens de sessão gerados com `secrets.token_urlsafe`, fornecendo entropia adequada.
- Passwords armazenadas com salt aleatório e comparação em tempo constante.
- Cookie de sessão com `HttpOnly` e `SameSite=Lax`.
- Respostas internas de erro não devolvem stack traces para o navegador.
- Escape centralizado de HTML no frontend para a maior parte dos dados apresentados.
- Filtros de tenant e associação de vendedor presentes nos fluxos comerciais principais.
- Controle de acesso específico para download de anexos de ocorrências.
- Limite agregado de 25 MB para anexos após decodificação.
- Banco em WAL, chaves estrangeiras habilitadas e busy timeout configurado.
- Service worker evita interceptar rotas iniciadas por `/api/`.

### Leitura equilibrada

Os achados críticos não significam que todo o sistema esteja inseguro. A base possui decisões corretas; o principal problema é a combinação de credenciais fixas, conteúdo não confiável servido no mesmo domínio e uma migração multiempresa incompleta.

## 6. Achados críticos

### CRIT-01 | CRÍTICA

#### Credencial fixa e redefinição automática do superadministrador

Evidência: backend/app.py, linhas 597-616; chamada durante migração e seed; README.md, seção de acessos de teste.

Impacto: A senha administrativa é conhecida por qualquer pessoa com acesso ao código ou documentação. Mesmo que seja alterada pelo usuário, volta ao valor fixo após reiniciar o backend. Com o login sem rate limiting, o risco de tomada completa da plataforma é imediato.

Recomendação: Remover a credencial do código, rotacionar a conta, criar bootstrap único por segredo externo, obrigar troca no primeiro acesso, invalidar sessões existentes e habilitar MFA para superadministradores.

### CRIT-02 | CRÍTICA

#### Stored XSS por anexos HTML ou SVG

Evidência: backend/app.py, linhas 1824-1832 e 2917-2931.

Impacto: O MIME declarado pelo cliente é armazenado sem validação e o arquivo é devolvido como inline no mesmo domínio. Um representante pode enviar HTML/SVG com JavaScript; quando um administrador abrir o anexo, o código executará na origem do HiperSales e poderá operar a API com a sessão administrativa.

Recomendação: Validar extensão, assinatura binária e MIME por allowlist; bloquear HTML/SVG; usar attachment para conteúdo não confiável; aplicar CSP sandbox/default-src none e preferir um domínio de arquivos sem cookies.

### CRIT-03 | CRÍTICA

#### Isolamento multiempresa incompleto

Evidência: backend/app.py, linhas 898-937, 3223-3245, 3453-3494, 6312-6335 e 6465-6471.

Impacto: O tenant é mantido em dicionário global por id da conexão e assume tenant 1 quando ausente. Configurações podem cair para valores globais, a senha SMTP pode ser exposta a tenants novos, a fila do WhatsApp não filtra tenant e rotinas de relatório usam consultas e destinatários globais.

Recomendação: Eliminar TENANT\_CONTEXT, exigir tenant\_id explícito e fail-closed, criar configurações por tenant, filtrar todas as filas e schedulers, remover destinatários fixos e criar testes automatizados de isolamento negativo.

## 7. Achados de severidade alta

### HIGH-01 | ALTA

#### Proteção insuficiente de login e sessão

Evidência: backend/app.py, linhas 1845-1918 e 4064-4085.

Impacto: O login aceita tentativas ilimitadas. O cookie não usa Secure, não existe MFA, mudança de senha não invalida sessões anteriores e `must_change_password` é principalmente uma barreira de interface. O status do tenant também não é verificado ao validar a sessão.

Recomendação: Adicionar rate limiting por IP e conta, MFA para perfis críticos, cookie `__Host-` com Secure, bloqueio server-side durante troca obrigatória, revogação de sessões e verificação de tenant ativo.

### HIGH-02 | ALTA

#### Gestão inadequada de segredos e artefatos sensíveis

Evidência: `.whatsapp-internal-token` não está no `.gitignore`; SMTP é armazenado no SQLite e retornado pela API; `CARGA_LEON.xlsx` é consumida com campo de senha em texto claro.

Impacto: Cópias do diretório, backups ou futuros commits podem expor tokens, credenciais SMTP, dados empresariais e senhas iniciais. A configuração global amplia o impacto para tenants adicionais.

Recomendação: Rotacionar segredos, usar secret manager ou variáveis protegidas, nunca devolver senha SMTP, remover planilhas após migração segura, ampliar `.gitignore` e adicionar varredura de segredos no CI.

### HIGH-03 | ALTA

#### Dependência transitiva sharp vulnerável

Evidência: whatsapp-service/package-lock.json: sharp 0.34.5. npm audit reportou advisory GHSA-wq5f-xc86-pv6w.

Impacto: A versão instalada está abaixo da 0.35.5 corrigida. O advisory classifica a vulnerabilidade como alta e descreve possibilidade de comprometimento de memória/RCE em condições específicas no Linux.

Recomendação: Atualizar Baileys ou aplicar override compatível para sharp  $\geq 0.35.5$ , reinstalar a partir do lockfile, repetir npm audit e validar conexão, QR, envio e recebimento de mensagens.

## 8. Achados de severidade média

### MED-01 | MÉDIA

#### Requisições sem limite global de tamanho

Evidência: backend/app.py, linhas 1789-1796.

Impacto: Content-Length é lido integralmente antes da validação. O endpoint de login também aceita corpos arbitrariamente grandes. Em conjunto com ThreadingHTTPServer, isso permite consumo excessivo de memória e threads.

Recomendação: Rejeitar corpos acima de limites por rota antes da leitura, configurar timeouts e limites no proxy e usar servidor WSGI/ASGI de produção.

### MED-02 | MÉDIA

#### Custo de hash de senha abaixo da referência atual

Evidência: backend/app.py, linhas 259-270: PBKDF2-HMAC-SHA256 com 150.000 iterações.

Impacto: O algoritmo possui salt e comparação correta, porém oferece proteção inferior à recomendação atual em caso de vazamento do banco.

Recomendação: Migrar progressivamente para Argon2id. Se PBKDF2 for obrigatório, elevar o custo para pelo menos 600.000 e rehash no próximo login, após benchmark de capacidade.

### MED-03 | MÉDIA

#### Injeção de HTML em mensagens de e-mail

Evidência: backend/app.py, linhas 1593-1597: placeholders são substituídos sem escape contextual.

Impacto: Nomes, observações e outros campos controlados por usuários podem alterar o HTML dos e-mails, inserir links ou elementos de rastreamento e produzir mensagens enganosas.

Recomendação: Escapar todos os valores por contexto HTML e permitir HTML apenas no template administrativo validado/sanitizado.

## 9. Outros riscos técnicos e operacionais

### MED-04 | MÉDIA

#### Cabeçalhos e servidor de produção incompletos

Evidência: backend/app.py, linhas 1724-1757 e 7672-7681.

Impacto: O servidor adiciona nosniff, mas não define CSP, frame-ancestors, HSTS, Referrer-Policy ou Permissions-Policy. Ele escuta em 0.0.0.0 e não implementa TLS diretamente.

Recomendação: Colocar o backend atrás de proxy HTTPS endurecido, impedir acesso direto à porta, configurar cabeçalhos de segurança e migrar a camada HTTP para framework/servidor mantido para produção.

### MED-05 | MÉDIA

#### Proteção CSRF baseada principalmente em SameSite

Evidência: Cookie SameSite=Lax; ausência de token CSRF e validação Origin/Referer; backend não valida Content-Type.

Impacto: SameSite reduz o risco, mas não cobre cenários same-site em subdomínios, browsers legados, ataques client-side ou futuras mudanças de implantação.

Recomendação: Validar Origin/Referer em operações mutáveis, exigir application/json, adicionar token ou header CSRF e manter SameSite como defesa adicional.

#### Referências de segurança

- OWASP Session Management Cheat Sheet — cookies Secure/HttpOnly/SameSite e ciclo de vida de sessão.
- OWASP CSRF Prevention Cheat Sheet — token, header customizado e validação de origem.
- OWASP Password Storage Cheat Sheet — Argon2id e parâmetros de PBKDF2.

## 10. Problemas funcionais e de arquitetura

---

- FUNC-01: O relatório chamado semanal usa período inicial de 365 dias em backend/app.py, linhas 4741-4746.
- FUNC-02: A rotina semanal seleciona representantes sem filtro de tenant e pode falhar ou operar apenas o tenant padrão.
- FUNC-03: customers.cnpj possui UNIQUE global, impedindo o mesmo CNPJ legítimo em tenants diferentes.
- FUNC-04: Evolution API e Baileys coexistem como dois caminhos de WhatsApp, aumentando estados divergentes e manutenção.
- FUNC-05: O banco contém sessões expiradas até que uma rota autenticada execute a limpeza.
- ENG-01: Backend com 7.685 linhas e frontend com 10.292 linhas concentram responsabilidades demais.
- ENG-02: Não foram encontrados testes próprios do projeto.
- ENG-03: Não existe requirements.txt ou pyproject.toml para reproduzir o ambiente Python.
- ENG-04: As migrações são executadas no startup sem versionamento, dry-run ou rollback formal.
- ENG-05: Há textos com mojibake, como sequências Ã e Â, indicando inconsistência de codificação.

### Consequência

A ausência de modularização e testes torna correções de segurança mais arriscadas, pois uma alteração em autenticação, tenant ou pedidos pode produzir regressões em áreas distantes do mesmo arquivo.

## 11. Avaliação do banco e dos dados

---

### Snapshot observado

A inspeção foi realizada em modo somente leitura, sem consultar ou expor registros individuais.

- Banco principal: database/hipersales.sqlite3, aproximadamente 10,3 MB.
- 21 tabelas de aplicação.
- 1 tenant ativo, 22 usuários, 308 clientes e 300 produtos.
- 208 propostas, 3 ocorrências e 2.090 registros de outbox.
- 11 sessões presentes, todas expiradas no momento da verificação.
- PRAGMA integrity\_check: ok.
- PRAGMA foreign\_key\_check: nenhuma violação.
- Journal mode: WAL.
- backend/hipersales.db existe com tamanho zero e aparenta ser artefato obsoleto.

### Riscos de proteção de dados

- Dados cadastrais, comunicações e anexos ficam sem criptografia no SQLite.
- O outbox retém destinatários, assunto, corpo e erros de envio.
- O form\_payload armazena cópia ampla de dados e resposta bruta da consulta de CNPJ.
- Backup do arquivo principal isolado pode perder alterações ainda presentes no WAL.
- É necessário definir retenção, acesso, backup, restauração e descarte compatíveis com a LGPD.

## 12. Validações realizadas

---

- Parsing AST de backend/app.py e scripts/import\_carga\_leon.py: aprovado.
- node --check em frontend/app.js: aprovado.
- node --check em whatsapp-service/index.mjs: aprovado.
- PRAGMA integrity\_check: aprovado.
- PRAGMA foreign\_key\_check: zero violações.
- Verificação de listener local: nenhuma aplicação escutava na porta 8000.
- npm ls: árvore principal resolvida com Baileys 7.0.0-rc13, Pino 9.14.0 e qrcode-terminal 0.12.0.
- npm audit: uma vulnerabilidade alta e uma moderada.
- protobufjs 7.6.2: advisory de DoS; versão corrigida indicada 7.6.5.
- sharp 0.34.5: advisory de alta severidade; versão corrigida indicada 0.35.5.

### O que não foi executado

- O backend não foi iniciado para evitar mutações, redefinição de senha e disparos automáticos.
- Não foram enviados e-mails, mensagens de WhatsApp ou requisições de teste aos provedores externos.
- Não houve exploração do XSS ou tentativa de autenticação com credenciais conhecidas.

## 13. Plano de correção priorizado

---

### Fase 0 — contenção imediata, 0 a 48 horas

- Rotacionar superadministrador, token interno, SMTP e credenciais relacionadas.
- Remover a redefinição automática e as credenciais do README.
- Bloquear upload/visualização inline de HTML e SVG.
- Restringir a aplicação a HTTPS e acesso controlado.
- Fazer backup consistente do SQLite com WAL e testar a cópia.
- Atualizar sharp e protobuffs por versão compatível do Baileys/overrides.

### Fase 1 — hotfix de segurança, até 15 dias

- Substituir TENANT\_CONTEXT por tenant explícito e fail-closed.
- Corrigir filas, relatórios, notificações e configurações por tenant.
- Adicionar rate limiting, política server-side de senha e invalidação de sessões.
- Adicionar validação Origin/CSRF, limites de corpo e cabeçalhos de segurança.
- Mascarar segredos nas respostas e externalizar sua persistência.
- Criar testes de autorização e isolamento para todas as rotas mutáveis.

## 14. Evolução em 30 a 90 dias

---

### De 15 a 45 dias

- Migrar hashes para Argon2id e implementar MFA para contas privilegiadas.
- Separar autenticação, tenants, clientes, pedidos, ocorrências, relatórios e integrações em módulos.
- Adotar framework web mantido, migrações versionadas e servidor de produção.
- Escolher uma única integração de WhatsApp e remover o caminho legado.
- Criar logs estruturados, auditoria de ações críticas e alertas operacionais.
- Corrigir o relatório semanal e parametrizar destinatários por tenant.

### De 45 a 90 dias

- Avaliar PostgreSQL para concorrência, constraints compostas e crescimento multiempresa.
- Implantar CI com testes, lint, auditoria de dependências e detecção de segredos.
- Implementar backups automáticos criptografados e testes periódicos de restauração.
- Definir políticas de retenção para anexos, outbox, sessões e payloads de terceiros.
- Executar teste de invasão independente após as correções críticas.
- Documentar resposta a incidentes e responsabilidades de acesso conforme LGPD.

## 15. Arquitetura alvo recomendada

### Entrada

Proxy HTTPS/WAF com rate limiting, limites de corpo, HSTS e cabeçalhos de segurança.

### Aplicação

Framework Python mantido, rotas modulares, middleware de autenticação/tenant e validação tipada.

### Dados

PostgreSQL ou SQLite com disciplina operacional, migrations versionadas, constraints por tenant e backups testados.

### Arquivos

Object storage ou domínio separado, download autenticado e conteúdo não confiável sempre isolado.

### Filas

Worker dedicado para e-mail, WhatsApp e relatórios, sempre com tenant\_id obrigatório e idempotência.

### Segredos

Secret manager/variáveis protegidas; a interface recebe apenas indicação de que o segredo está configurado.

### Princípio central

Nenhuma função deve descobrir o tenant por fallback ou estado global. O tenant precisa ser derivado da sessão validada, passado explicitamente até o repositório de dados e incluído em todas as chaves, consultas, filas, logs e testes.

## 16. Critérios mínimos para produção

---

- ? Não existe senha, token ou chave fixa no código, documentação ou planilha versionável.
- ? Reiniciar a aplicação não modifica senhas nem dados de negócio.
- ? Anexos maliciosos não executam código na origem do HiperSales.
- ? Um tenant não consegue ler, alterar, associar, enviar ou inferir dados de outro tenant.
- ? Tenant suspenso não autentica e perde acesso em sessões existentes.
- ? Login possui limitação, monitoração e MFA para contas privilegiadas.
- ? Troca de senha revoga sessões anteriores e é imposta pelo backend.
- ? Cookie usa Secure, HttpOnly, SameSite e prefixo \_\_Host- em HTTPS.
- ? Todas as operações mutáveis possuem proteção CSRF/origin.
- ? npm audit não apresenta vulnerabilidade alta ou crítica aplicável.
- ? Testes automatizados cobrem autorização, tenant, anexos, pedidos e integrações.
- ? Backup e restauração foram executados com sucesso em ambiente separado.
- ? Logs de auditoria permitem identificar autor, tenant, ação, data e resultado.

## 17. Mapa resumido de evidências

---

- backend/app.py:259 — PBKDF2-HMAC-SHA256 com 150.000 iterações.
- backend/app.py:358-364 — unicidade global de CNPJ.
- backend/app.py:597-616 — criação/redefinição do superadministrador.
- backend/app.py:898-937 — fallback de configurações e tenant 1.
- backend/app.py:1593-1597 — substituição de template sem escape HTML.
- backend/app.py:1724-1796 — servidor HTTP e leitura integral do corpo.
- backend/app.py:1824-1832 — arquivos servidos inline pelo MIME informado.
- backend/app.py:1845-1918 — sessão e cookie sem Secure.
- backend/app.py:2917-2931 — upload aceita MIME do cliente.
- backend/app.py:3223-3245 — resposta inclui configuração SMTP.
- backend/app.py:3453-3494 — fila WhatsApp sem tenant.
- backend/app.py:4741-4746 — relatório semanal com 365 dias.
- backend/app.py:6312-6335 — scheduler semanal sem escopo de tenant.
- backend/app.py:6465-6471 — associação legada sem validação de tenant.
- backend/app.py:7672-7681 — schedulers e ThreadingHTTPServer.
- frontend/app.js:778-795 — escape HTML centralizado.
- whatsapp-service/index.mjs:44-57 — token interno em todas as chamadas.
- scripts/import\_carga\_leon.py:107-128 — importação redefine senhas.
- whatsapp-service/package-lock.json:1046 e 1133 — protobufjs e sharp vulneráveis.

## 18. Conclusão

O HiperSales possui valor funcional real e uma base capaz de evoluir. Os fluxos de clientes, produtos, pedidos, ocorrências e relatórios estão amplamente implementados, e há bons controles locais de SQL, sessões aleatórias, escape de interface e integridade do banco.

Entretanto, três condições impedem uma recomendação de produção pública no estado atual: a credencial fixa do superadministrador, a possibilidade de execução de JavaScript por anexos e o isolamento multiempresa incompleto. Esses pontos podem resultar em comprometimento administrativo, vazamento de dados e envio de informações pelo canal errado.

A recomendação é executar primeiro a contenção e o hotfix de segurança, preservando o escopo funcional. Depois, modularizar gradualmente o sistema com testes de autorização e tenant. Essa sequência reduz risco sem exigir uma reescrita imediata.

### Parecer técnico final

A aplicação pode continuar em uso controlado após contenção operacional, mas requer correções críticas antes de exposição pública ou ativação de múltiplos tenants. Uma nova revisão deve ser realizada após o hotfix e antes da liberação em produção.

**Miguel Alameida**

Desenvolvedor Nível 3

6 de outubro de 2026